

Tutorial de Arduino

Índice

Índice	1
1. Aprendizaje de la programación	2
Antes de empezar: notas importantes	2
1.1 Instalar el IDE de Arduino	2
1.2 Instalación del entorno de desarrollo ESP32 para el IDE de Arduino	4
1.3 Instalación del archivo de biblioteca	7
1.4 Instalar el controlador USB CH340	8
Instalar el controlador en un sistema Windows	9
Instalación del controlador en un sistema Linux	9
Instalación del controlador en un sistema Mac	10
1.5 Ejemplos de código	11
1.5.1 Zumbador integrado	11
1.5.2 Para controlar un servo	14
1.5.3 Lectura del valor del joystick	16
1.5.4 Robot controlado por joystick (con función de grabación y repetición de movimientos)	17
1.5.5 Lectura del valor de la aplicación web	20
1.5.6 Uso del brazo robótico	22
2. Grabar el firmware	23
2.1 Herramienta de grabación	24
2.2 Firmware de eArm (firmware de fábrica)	25
2.2.1 Grabar el firmware de eArm	25
2.3 Firmware de control mediante joystick (con función de grabación)	27
2.3.1 Grabación del firmware del mando de control	28
3. Control de calidad	30
3.1 No se reconocen los puertos serie USB	30
3.2 El brazo robótico no funciona	30
3.3 Otros problemas	30
4. Contacto	31

1

Aprendizaje de la programación

Antes de empezar: Notas importantes

La placa de control ESP32 viene con un firmware preinstalado que te permite manejar el brazo robótico tal y como se describe en la Guía de montaje y uso. El firmware preinstalado permanecerá en la placa hasta que cargues un nuevo código.

Notas importantes antes de cargar el código:

■ 1: La placa ESP32 siempre ejecuta el programa cargado más recientemente.

■ 2: Cada nueva carga sobrescribe el programa cargado anteriormente.

Este capítulo te guiará a través de la programación del robot utilizando ejemplos de código que van desde lo básico hasta lo avanzado. El ejemplo final incluye el código fuente del firmware preinstalado de fábrica, lo que te permite restaurar la funcionalidad original del brazo robótico en cualquier momento.

1.1 Instalar el IDE de Arduino

La última versión del IDE de Arduino se puede descargar desde la página web oficial de Arduino: <https://www.arduino.cc/en/software>

A continuación, mostraremos cómo instalar el IDE de **64 bits para Windows 10 y versiones posteriores** en un ordenador.



Arduino IDE 2.3.2

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits

Windows MSI installer

Windows ZIP file

Linux AppImage 64 bits (X86-64)

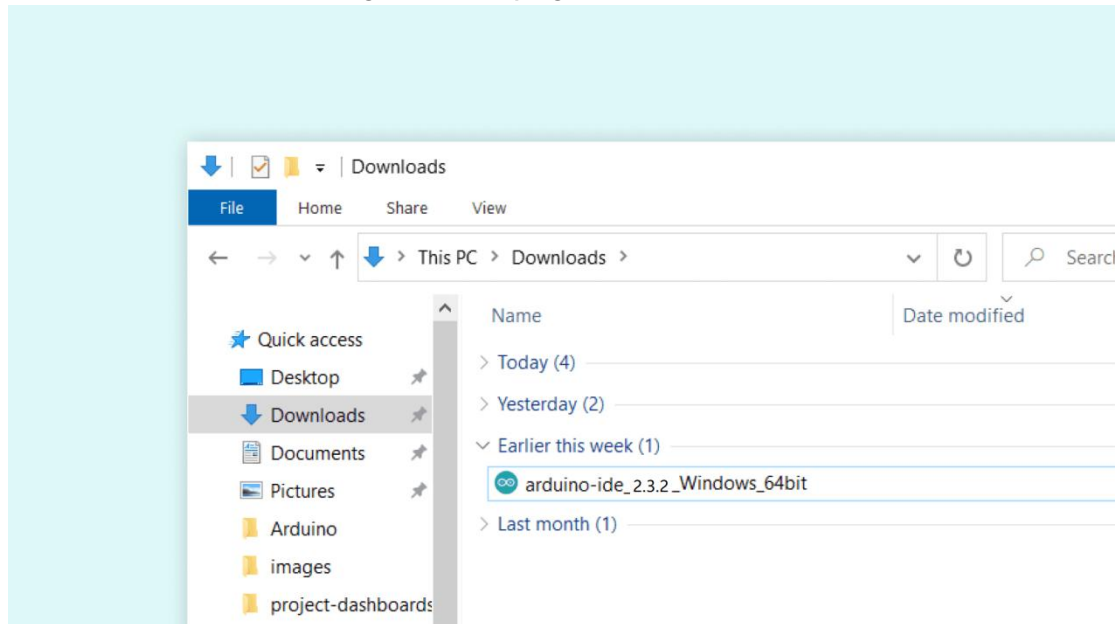
Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits

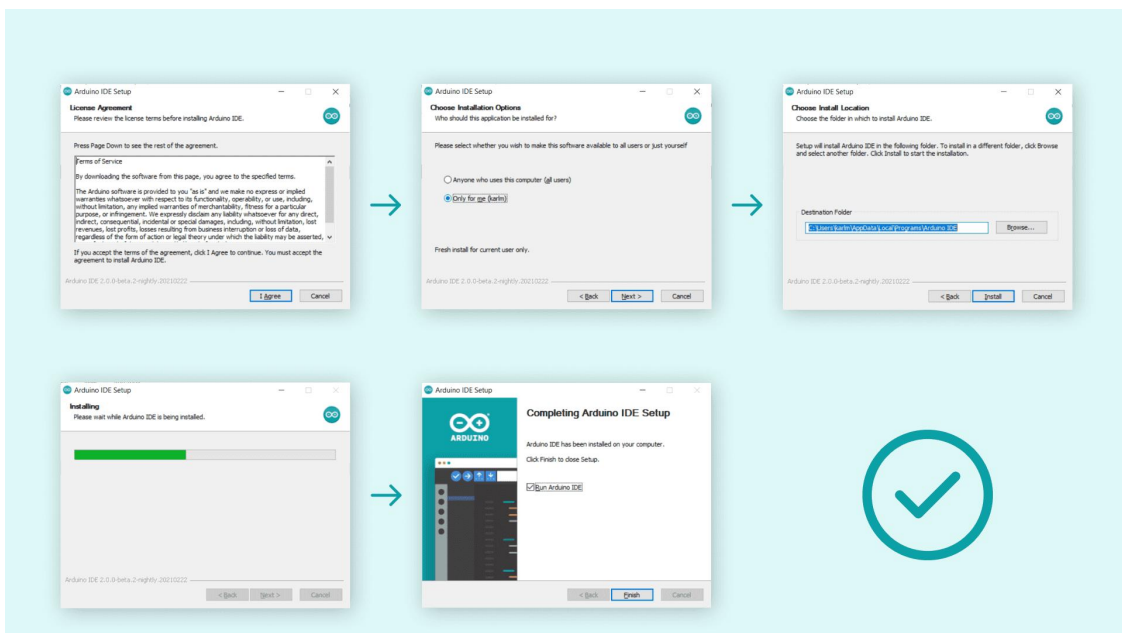
macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

Para instalar el IDE de Arduino 2.x.x en un ordenador con Windows, basta con ejecutar el archivo descargado de la página de software.



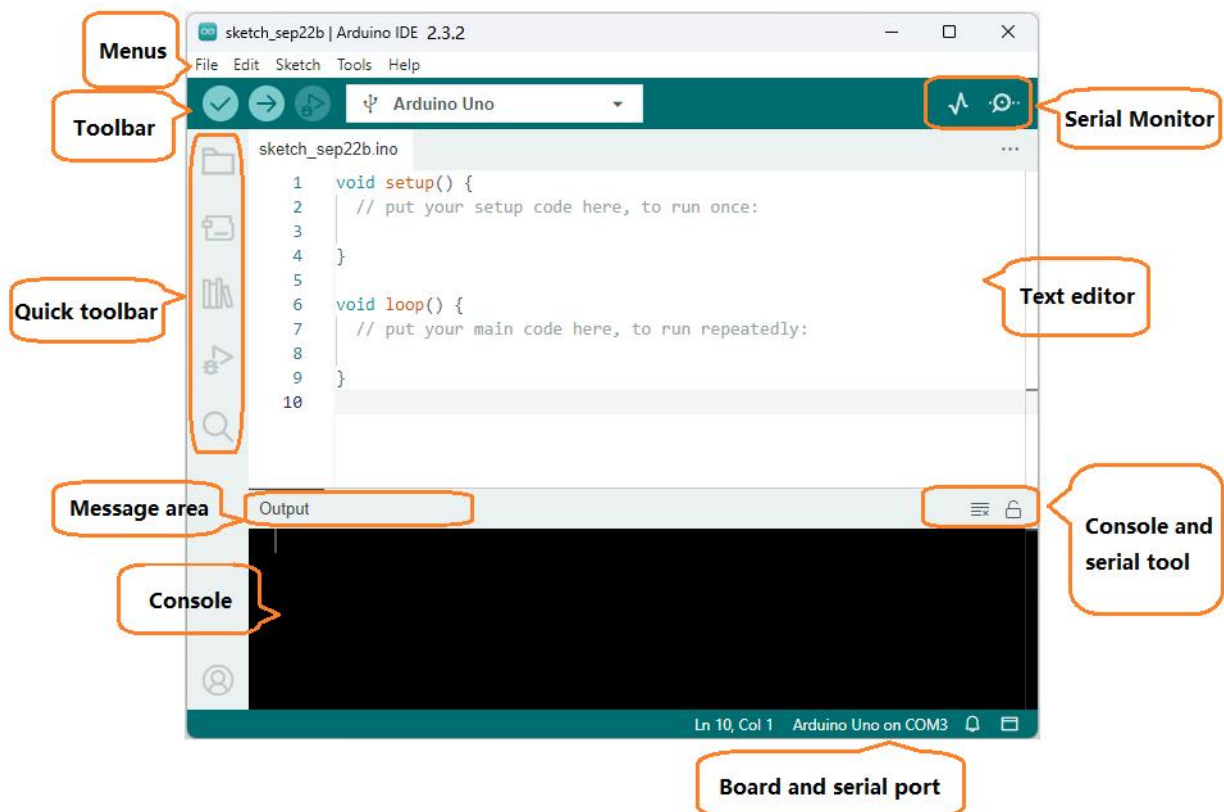
Sigue las instrucciones que aparecen en la ventana emergente para completar el proceso de instalación.



Icono de inicio:

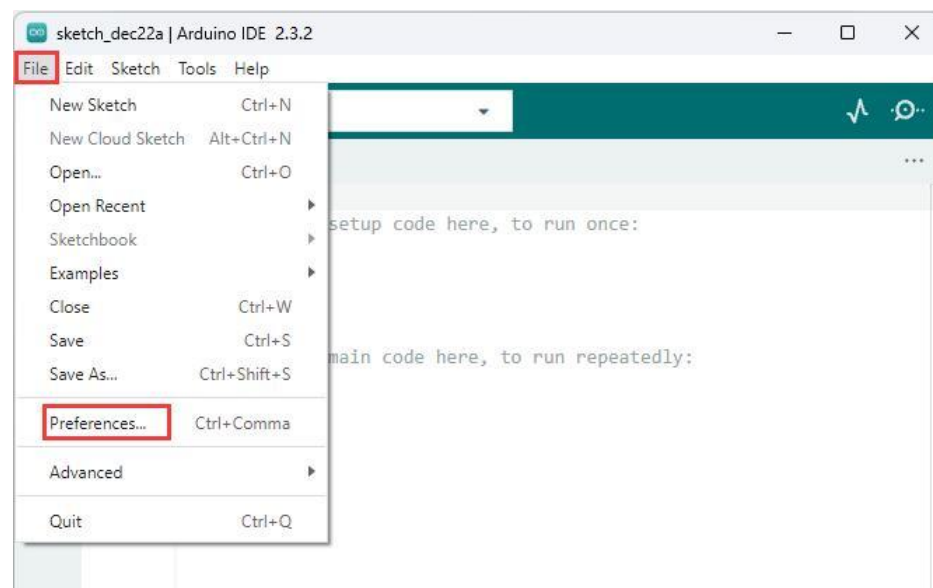


Página de inicio del IDE de Arduino



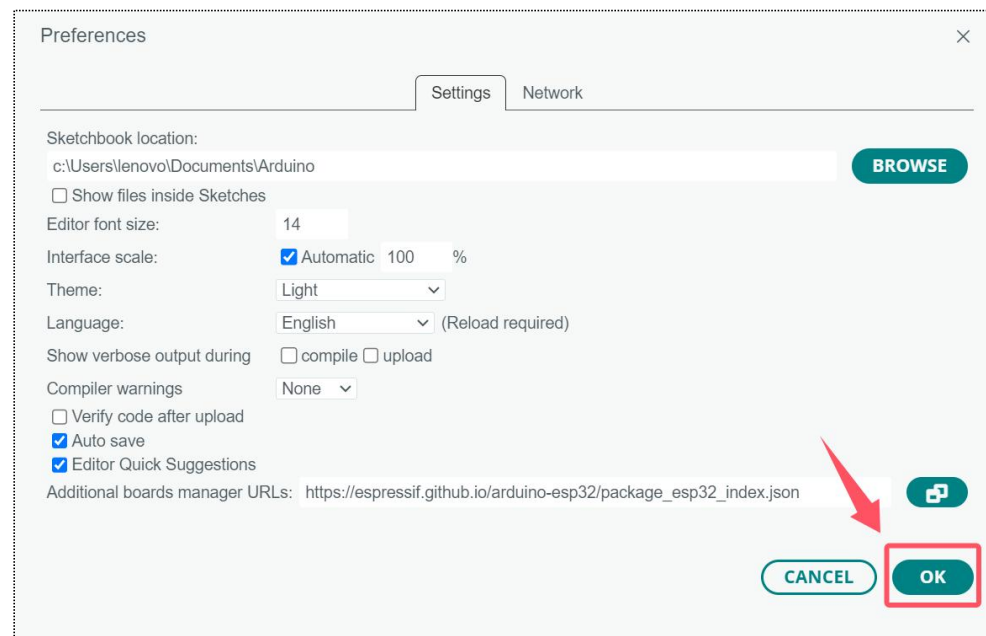
1.2 Instalar el entorno de desarrollo ESP32 para el IDE de Arduino

Abre el IDE de Arduino, haz clic en «Archivo» > «Preferencias», tal y como se muestra a continuación:



Desplázate hacia abajo en la ventana emergente y copia el enlace que aparece a continuación en el cuadro de texto «URL adicionales del gestor de placas». A continuación, haz clic en «Aceptar».

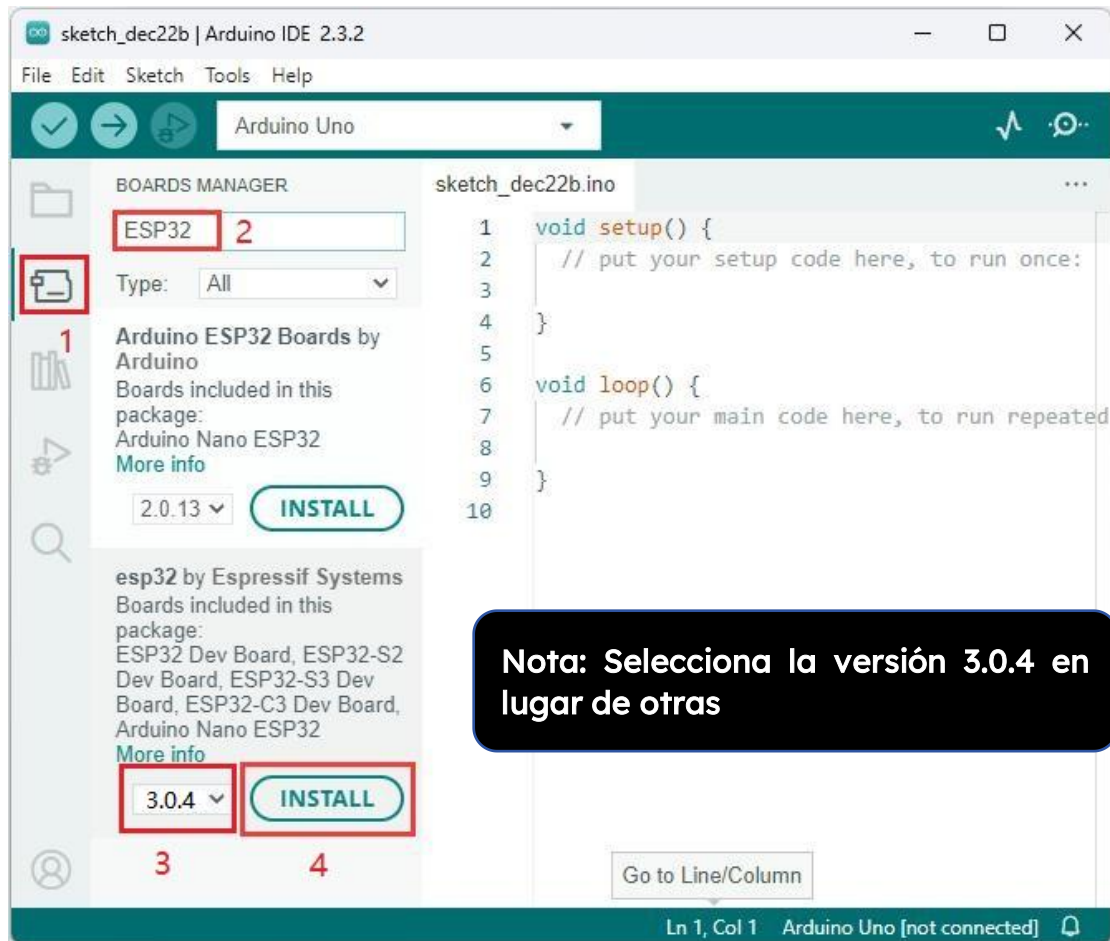
https://espressif.github.io/arduino-esp32/package_esp32_index.json



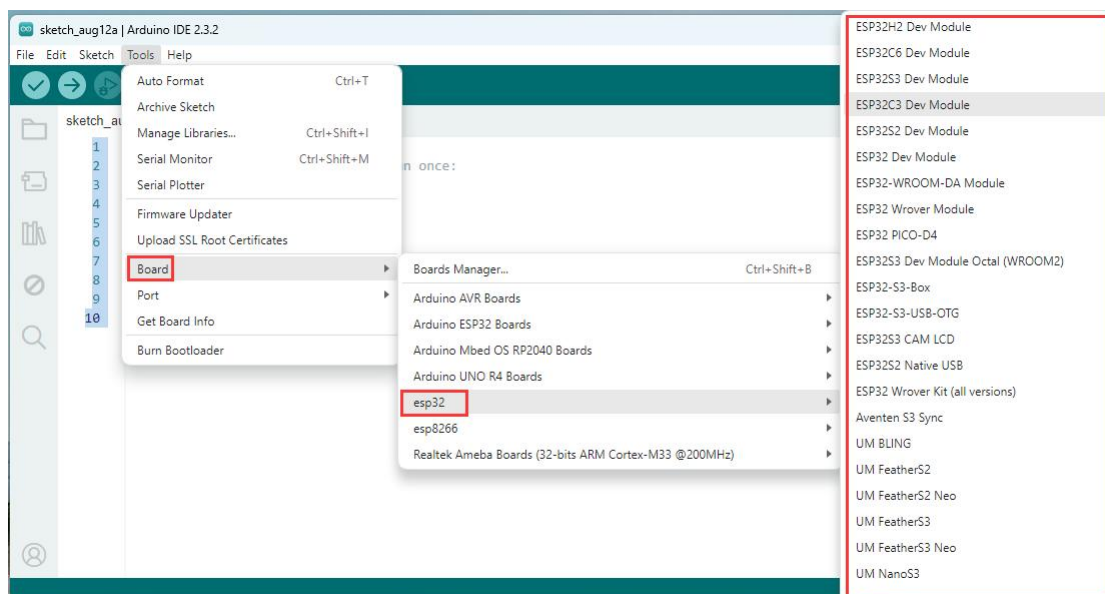
Haz clic en «Gestor de placas»:



Busca «ESP32» en el Gestor de placas, selecciona «esp32» de Espressif Systems en la versión 3.0.4 y haz clic en «INSTALAR»



Una vez instalada correctamente la placa ESP32, haz clic en el menú «Herramientas» -> «Placa» -> «ESP32» y selecciona «Módulo de desarrollo ESP32C3».



1.3 Instalar el archivo de biblioteca

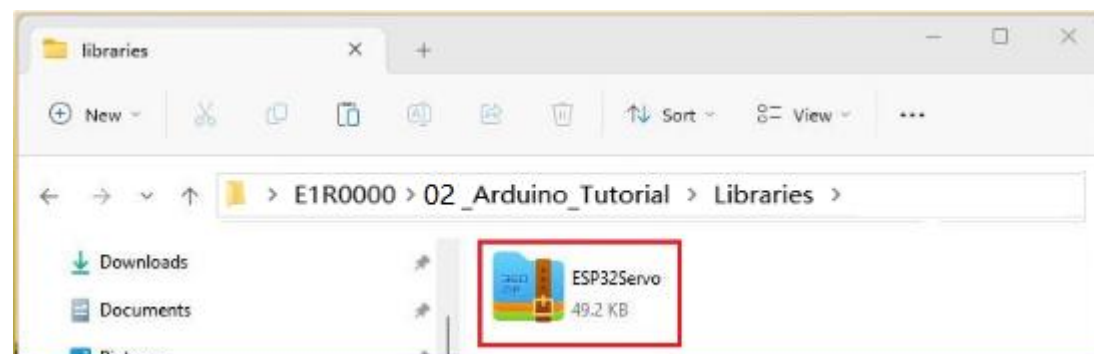
Método 1:

Abre el gestor de bibliotecas del IDE de Arduino y escribe «**ESP32Servo**» en la barra de búsqueda, selecciona el archivo de biblioteca denominado ESP32Servo y la versión **3.0.5**, y haz clic en «Instalar».

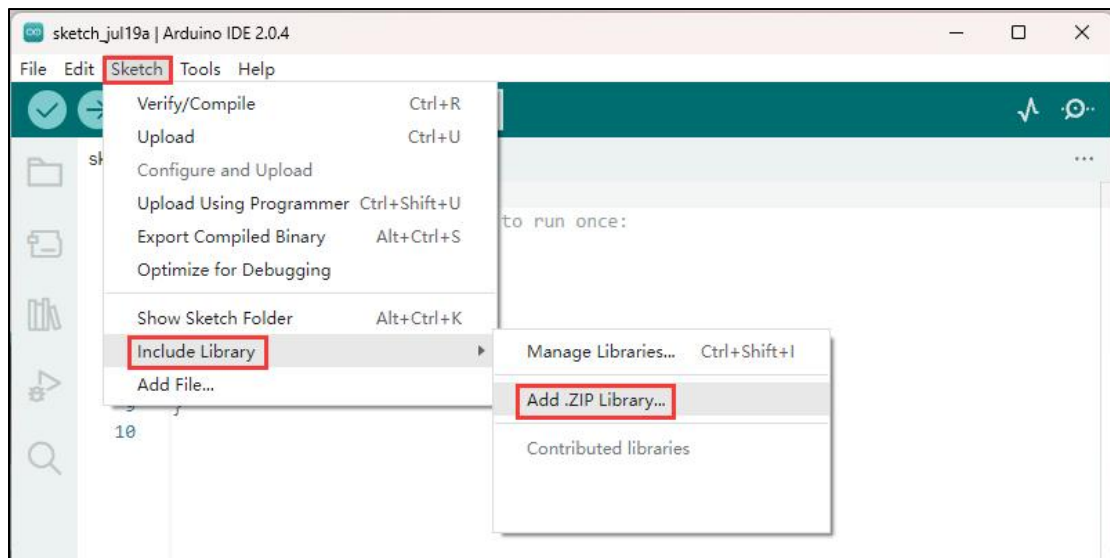


Método 2:

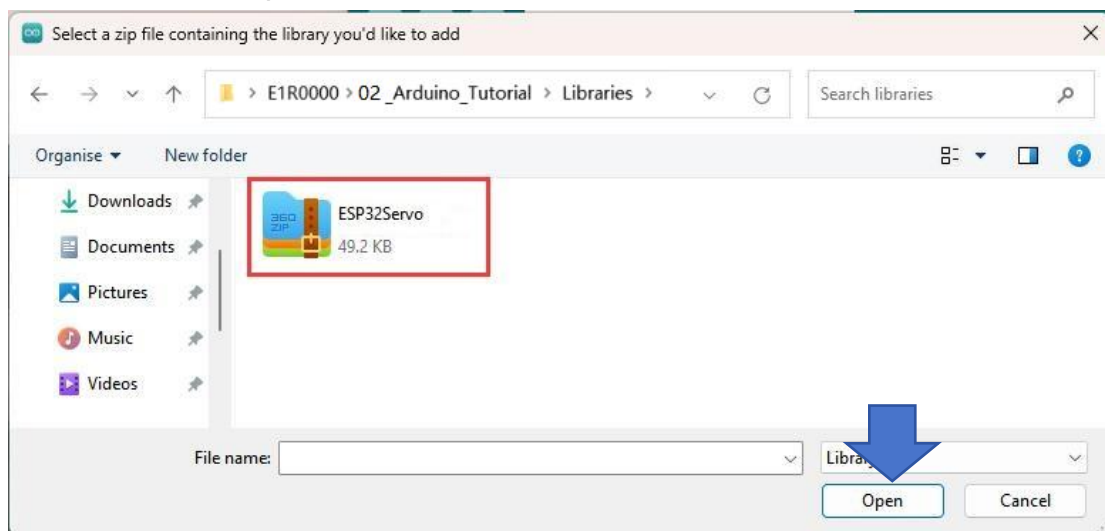
En el paquete del tutorial se incluye un archivo ZIP con la biblioteca, que se encuentra en la siguiente carpeta:



Importa el archivo ZIP de la biblioteca desde el IDE de Arduino de la siguiente manera:

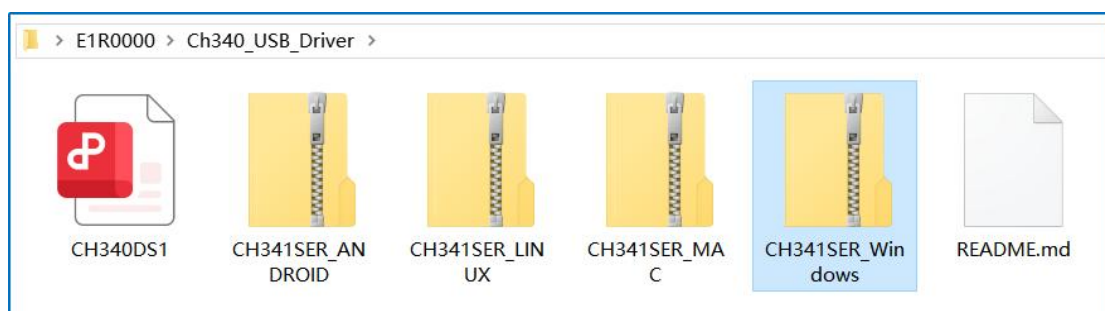


Haz clic en «Abrir» para instalar el archivo de la biblioteca.



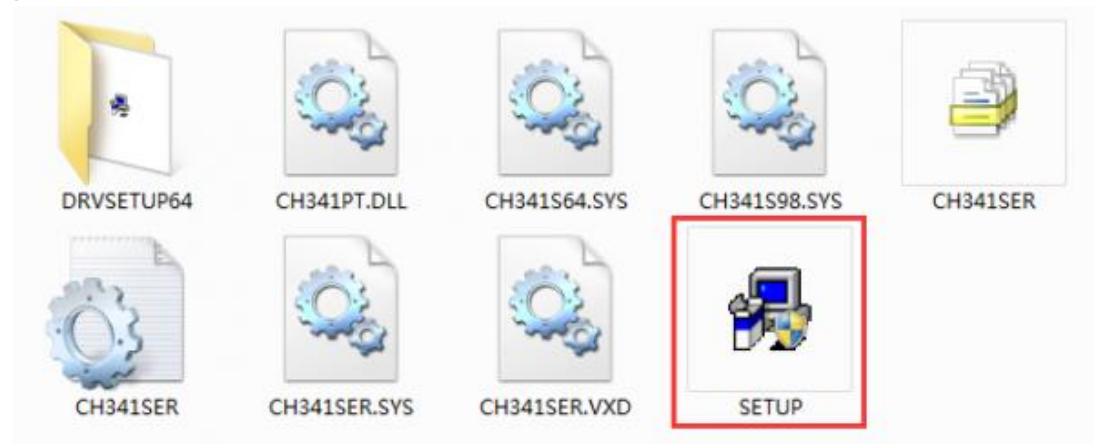
1.4 Instalar el controlador USB CH340

El archivo del controlador se incluye en nuestro paquete del tutorial:



Instala el controlador en un sistema Windows

Descomprime el archivo que te hemos proporcionado y haz doble clic en **«SETUP»** para ejecutar el instalador:



Si el controlador se instala correctamente, cuando la placa ESP32 esté conectada al ordenador mediante el cable USB, en la ruta **«Equipo» -> «Propiedades» -> «Administrador de dispositivos»**, podrás encontrar **«USB-SERIAL CH340 (COM3)»**, tal y como se muestra a continuación:

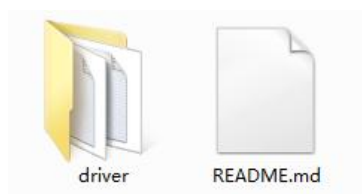


Nota: El símbolo que aparece para el controlador es **«USB-SERIAL CH340 (COMx)»**, donde «x» puede ser cualquier número, ya que depende del número que tu ordenador asigne al dispositivo ESP32.

Instalación del controlador en un sistema Linux

Es casi seguro que los controladores ya estén integrados en tu kernel de Linux y que funcionen nada más conectarlo. Si no es así, puedes descargar el controlador CH340 para Linux (aunque te recomiendo simplemente actualizar tu instalación de Linux para disponer del controlador «integrado»).

Si tienes que instalarlo tú mismo, consulta el archivo **«README.md»** que se incluye en el paquete zip.



Instalar el controlador en un sistema Mac

Enlace para descargar el controlador:

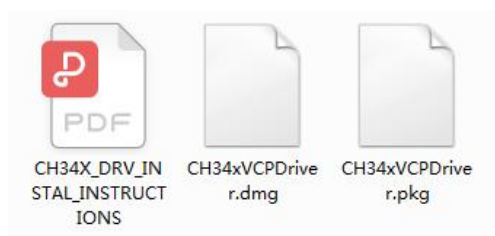
http://www.wch-ic.com/downloads/CH341SER_EXE.html

 download

relation files

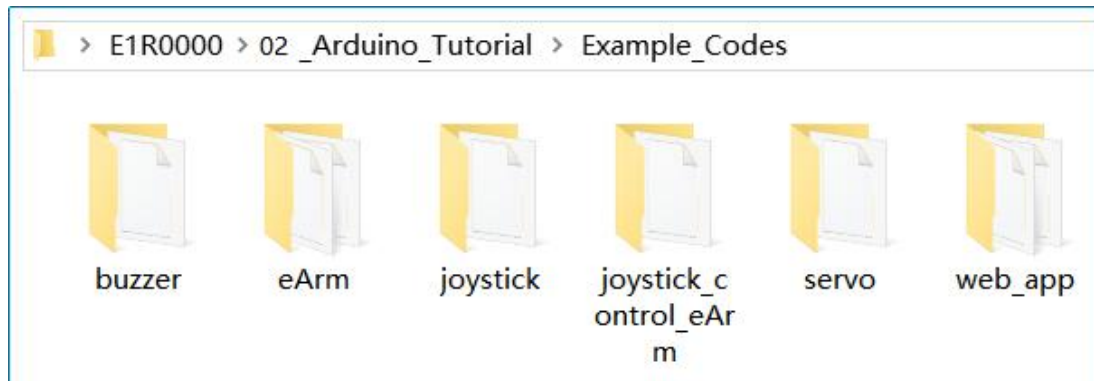
file name	file content
CH341SER.ZIP	CH340/CH341 USB to serial port Windows driver, supports Windows XP/Vista/7/8/8.1/10/11/ SERVER 2003/2008/2012/2016/2019/2022 -32/64bit, Microsoft WHQL Certified, supports USB to 3-line and 9-line serial port.
CH341SER_LINUX.ZIP	Linux driver for USB to serial port, supports CH340 and CH341, supports 32/64-bit operating systems.
CH341SER_MAC.ZIP	For CH340/CH341/CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9104/CH9143, USB to serial port VCP vendor driver of macOS , supports OS X 10.9~10.15 , OS X 11 (Big Sur) and above , contains installation guide documents.
CH340DS1.PDF	CH340 datasheet, USB bus converter chip which converts USB to serial port, printer port, IrDA SIR etc. Integrated clock, supports various operating systems, chip information can be customized, this datasheet is about USB to serial port and USB to Infrared Adapter SIR.
CH341DS1.PDF	CH341 datasheet, USB bus converter chip with multiple communication interfaces, such as USB to serial port/parallel port/printer port/I2C interface etc. Drivers support for Windows/Linux/Android/Mac, etc. The datasheet is the description of USB to serial port and printer port.
CH341SER_ANDROID.ZIP	CH340/CH341/CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9104/CH9143 USB to serial port Android driver-free application library and software, for Android OS 4.4 and above in USB Host mode, without loading Android kernel driver and without root access operation. Includes apk installer, lib library file (Java Driver), App Demo routines (USB to UART Demo project SDK).

Descarga el controlador de la página web y descomprime el archivo en un directorio de instalación local. Obtendrás una guía de instalación en PDF y dos paquetes de instalación en diferentes formatos. Para más detalles, consulta la guía de instalación en PDF.



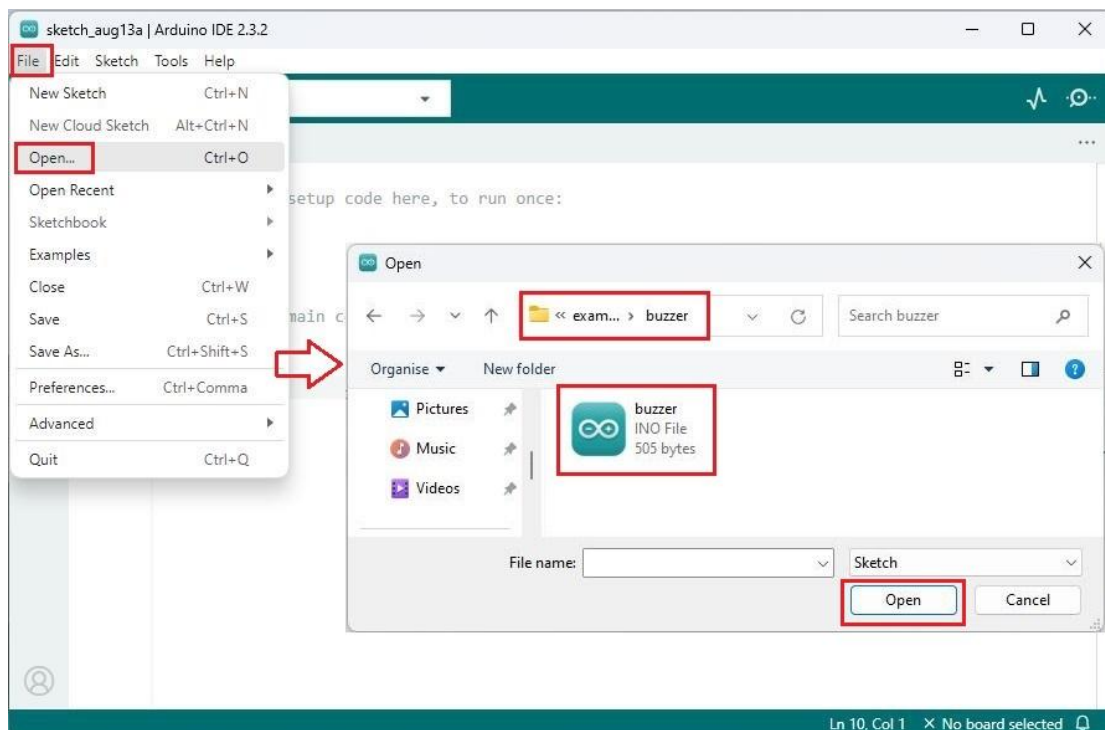
1.5 Ejemplos de código

Los códigos de ejemplo se encuentran en las siguientes carpetas del paquete del tutorial:

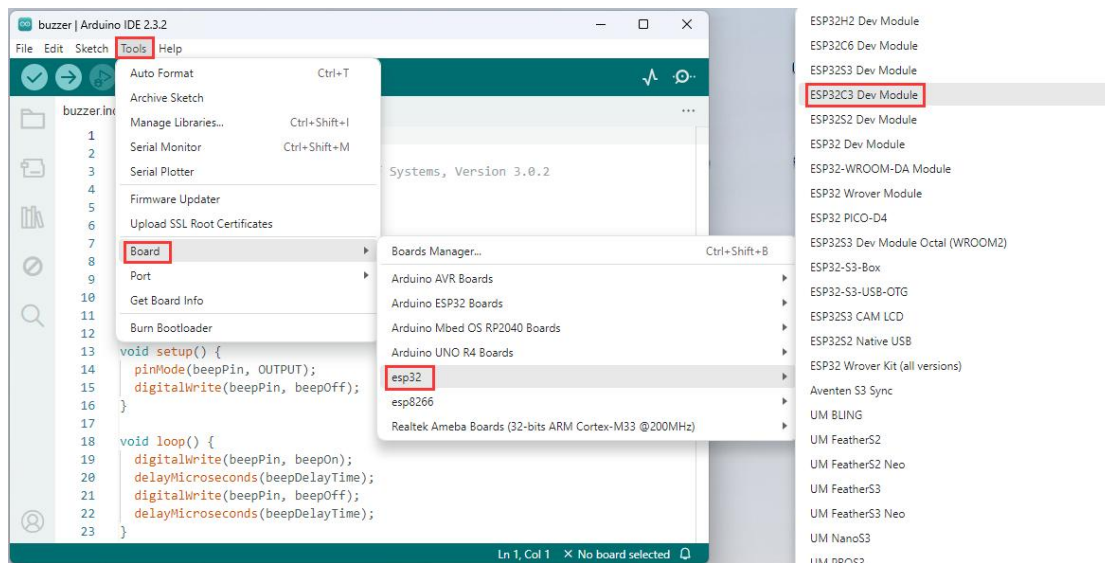


1.5.1 Zumbador integrado

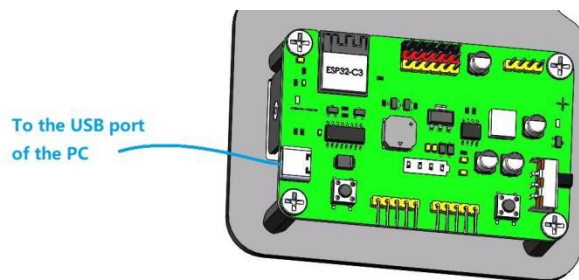
Abre el código de ejemplo «buzzer» en el IDE de Arduino:



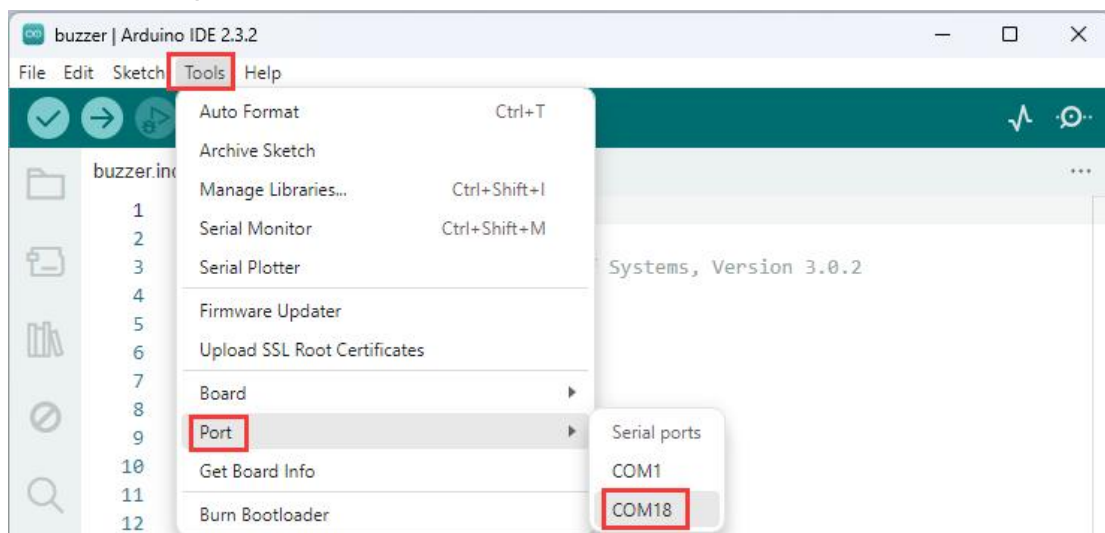
Selecciona el tipo de placa (**ESP32C3 Dev Module**):



Utiliza el cable USB para conectar el ordenador y el eArm, tal y como se muestra a continuación:

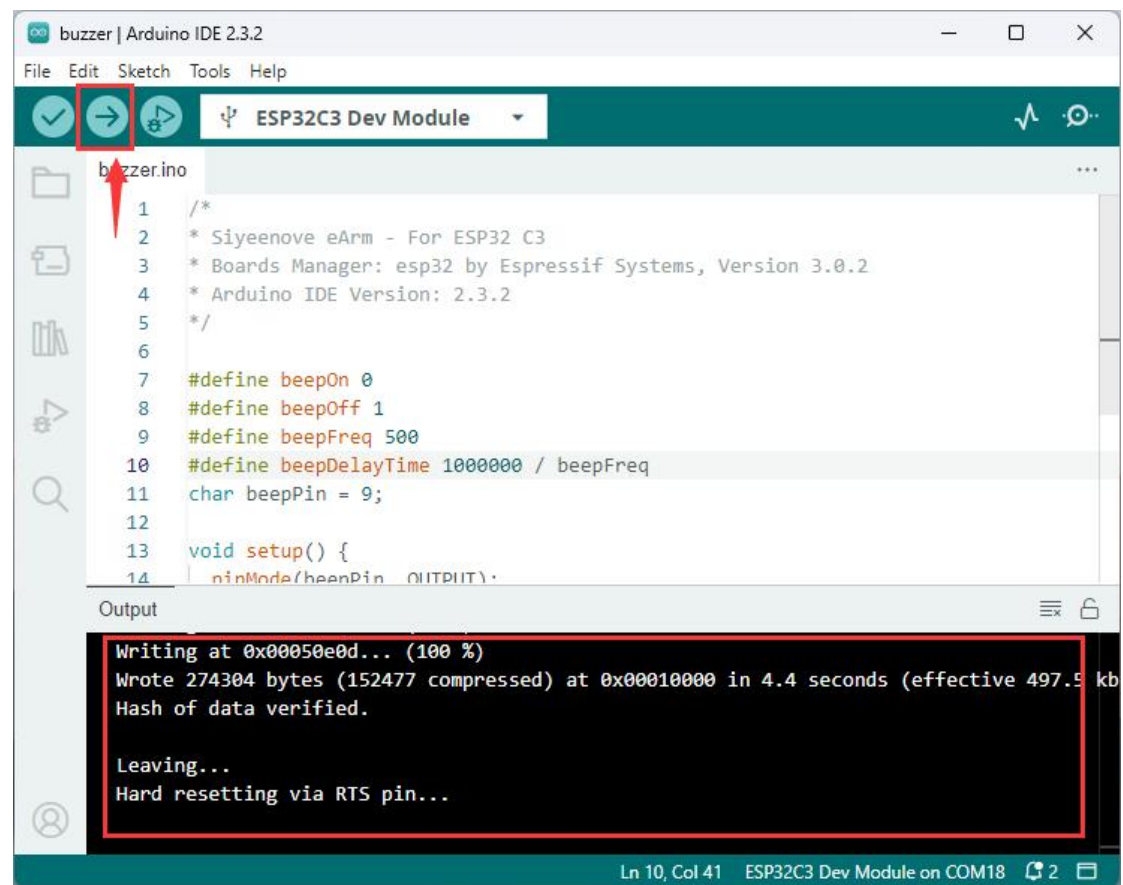


Selecciona el puerto:

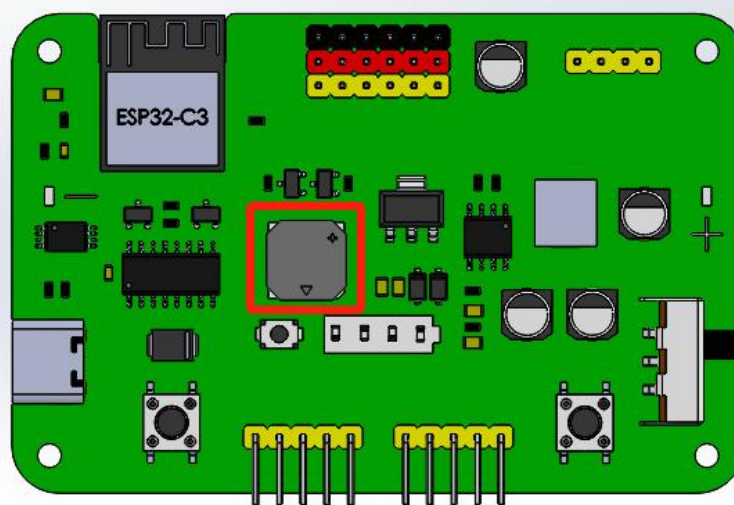


Nota: El puerto de la imagen anterior es el COM18. Es posible que tu ordenador tenga un puerto con un número diferente. Selecciónalo según el puerto asignado por el administrador de dispositivos de tu ordenador.

Haz clic para **cargar** el código:



Una vez cargado el código correctamente, el zumbador de la placa ESP32 seguirá emitiendo sonidos:

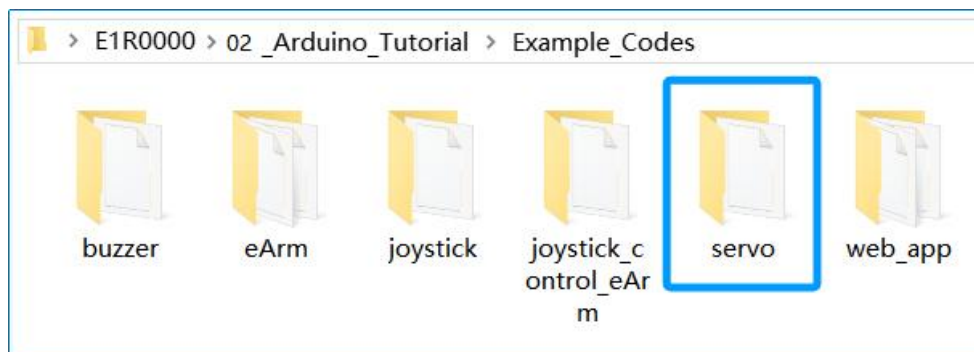


Explicación del código:

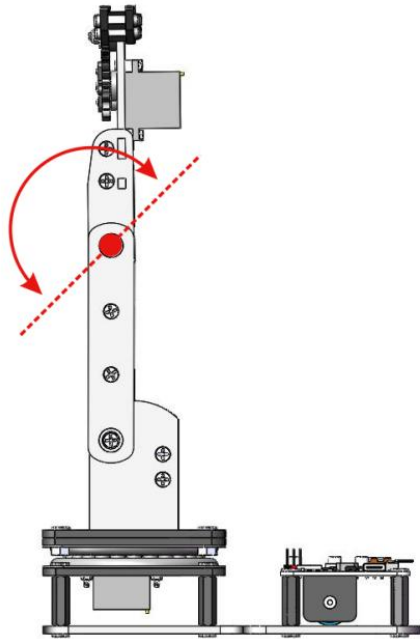
<code>#define beepOn 0</code>	Define una constante «beepOn» con el valor 0.
<code>#define beepDelayTime 1000000 / beepFreq</code>	Define una constante «beepDelayTime» con el valor: 1000000 / beepFreq.
<code>char beepPin = 9;</code>	Define una variable de tipo carácter «beepPin» con el valor 9, asignada al pin de control del zumbador.
<code>void setup() { ... }</code>	Función integrada del IDE de Arduino, que solo se ejecuta una vez al encender el dispositivo.
<code>pinMode(beepPin, OUTPUT);</code>	Configurar el pin 9 en modo de salida.
<code>digitalWrite(beepPin, beepOff);</code>	El pin 9 emite un nivel alto para apagar el zumbador.
<code>void loop() { ... }</code>	Función integrada del IDE de Arduino que se ejecuta en bucle de forma continua mientras el dispositivo está encendido.
<code>delayMicroseconds(beepDelayTime);</code>	Función integrada del IDE de Arduino; retardo en microsegundos.

1.5.2 Para controlar un servo

Sigue las instrucciones del capítulo 1.5.1 para cargar el código «servo» en la placa de control esp32:



Una vez cargado el código correctamente, el servo C del eArm oscilará entre 0 y 180, y de 180 a 0 en un ciclo:

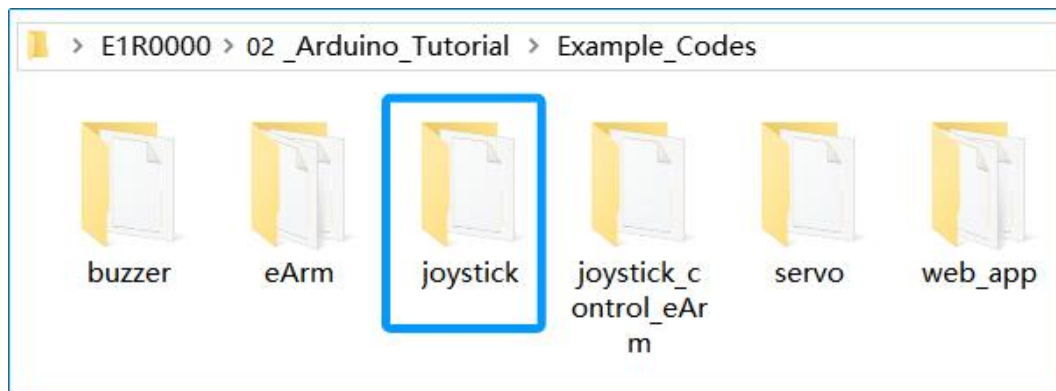


Explicación del código:

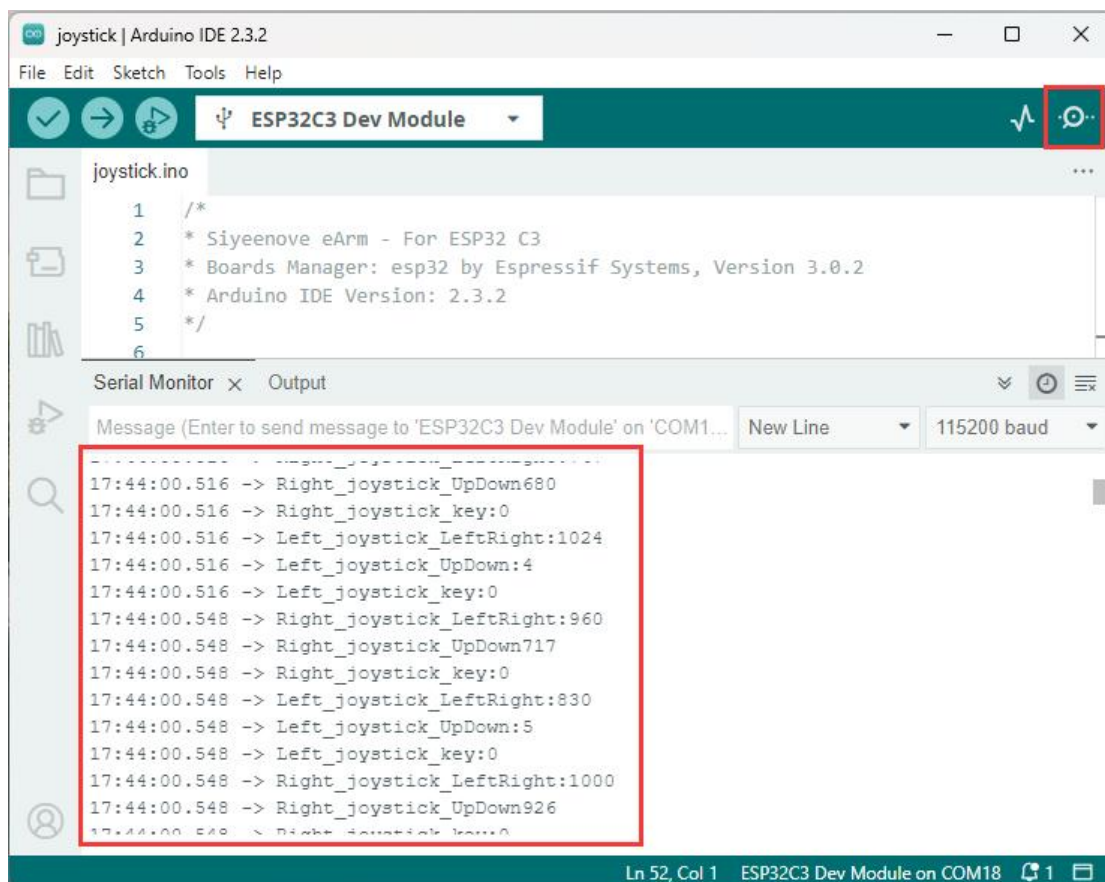
<code>#include <ESP32Servo.h></code>	Incluye una biblioteca de controladores de servo para ESP32.
<code>Servo Aservo;</code>	Define una clase de servo para controlar el Servo1.
<code>ESP32PWM::allocateTimer(0);</code>	Permitir que la biblioteca del controlador del servo utilice el temporizador 0.
<code>Aservo.setPeriodHertz(50);</code>	Establece la frecuencia de los impulsos del servo Aservo en 50 Hz.
<code>Aservo.attach(AservoPin, 500, 2400);</code>	Configura el servo de accionamiento «Aservo» para que utilice el pin «AservoPin» para el accionamiento, con una duración de pulso mínima de 500 microsegundos y una duración de pulso máxima de 2400 microsegundos.
<code>Aservo.write(aServoAngle);</code>	Configura el servo de accionamiento Aservo para que gire hasta el ángulo aServoAngle.
<code>for(instrucción1; condición; instrucción2){ ... }</code>	En el bucle «for», cada vez que se ejecuta la instrucción 1, se comprueba si la condición es verdadera. Si es así, se ejecutan las instrucciones entre llaves; de lo contrario, el bucle se interrumpe. A continuación, se ejecuta la instrucción 2, se vuelve a comprobar la condición y se repite el bucle.

1.5.3 Leer el valor del joystick

Sigue las instrucciones del capítulo 1.5.1 para cargar el código del «joystick» en la placa de control ESP32:



A continuación, abre el monitor de puerto serie del IDE de Arduino, mueve la palanca hacia la izquierda, la derecha, arriba o abajo, o pulsa el joystick hacia abajo, y el puerto serie mostrará el valor del joystick:

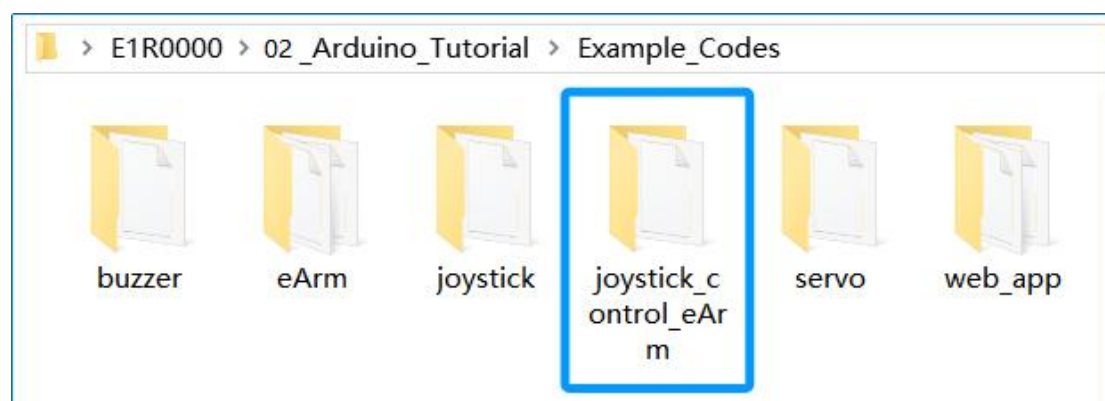


Explicación del código:

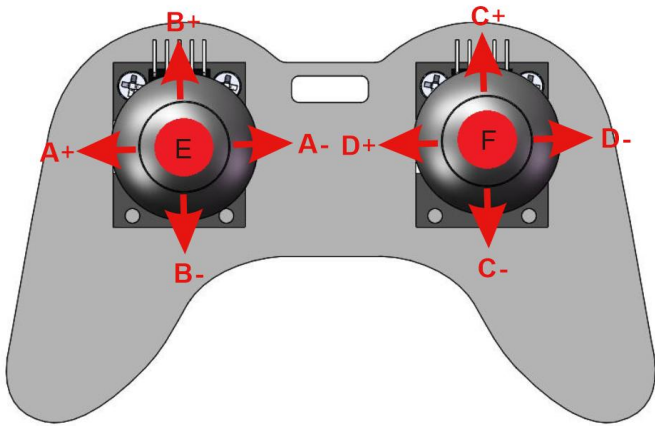
<code>Serial.begin(115200);</code>	Establece la velocidad de transmisión del puerto serie en 115200
<code>pinMode(left_joystick_KeyPin, INPUT);</code>	Establece el pin de la tecla del joystick izquierdo como «left_joystick_KeyPin» y el modo en salida.
<code>pinMode(right_joystick_KeyPin, INPUT_PULLUP);</code>	Configurar el pin de la tecla del joystick derecho como «right_joystick_KeyPin», el modo como salida y con una resistencia pull-up.
<code>Serial.print("Left_joystick_LeftRight:");</code>	El puerto serie imprime una cadena sin saltos de línea.
<code>Serial.println(analogRead(left_joystick_LeftRightPin));</code>	El puerto serie imprime valores analógicos y realiza un salto de línea.
<code>analogRead(left_joystick_LeftRightPin);</code>	Lee el valor del joystick izquierdo al moverlo hacia la izquierda y hacia la derecha.
<code>delay(1000);</code>	Retraso de 1000 milisegundos.

1.5.4 Robot controlado por joystick (con función de grabación y repetición de movimientos)

Sigue las instrucciones del capítulo 1.5.1 para cargar el código «joystick_control_eArm» en la placa de control esp32:



Control mediante joystick – Guía de funcionamiento del brazo robótico:

	
A+: Controla el servomotor A; el brazo robótico gira hacia la izquierda.	A-: Controla el servomotor A; el brazo robótico gira a la derecha.
B+: controla el servomotor B; el brazo robótico se balancea hacia delante.	B-: Controla el servomotor B; el brazo grande del brazo robótico se balancea hacia atrás.
C+: Controla el servomotor C; el brazo pequeño del brazo robótico se balancea hacia delante.	C-: Controla el servomotor C; el brazo pequeño del robot se balancea hacia atrás.
D+: Controla el servomotor D; la pinza del brazo robótico se abre (se afloja).	D-: Controla el servo D; la pinza del brazo robótico se cierra (se aprieta).
E: Grabar acción – Pulsa el botón E una vez para grabar una acción (se pueden grabar hasta 20 acciones seguidas). Cada vez que se pulsa, se oye un pitido corto. Cuando se alcanza la capacidad máxima de grabación, el zumbador emite un pitido largo.	F: Reproducción — Tras grabar la secuencia, pulsa una vez el botón F para iniciar la reproducción en bucle de las acciones grabadas de forma continua. El zumbador emitirá un pitido y el brazo repetirá las acciones en bucle. Para salir del bucle, mantén pulsado el botón F (pulsación prolongada); el zumbador emitirá un pitido largo para indicar que la reproducción se ha detenido.

Nota: ¡ Dado que se utiliza memoria dinámica, el eArm perderá todas las acciones grabadas tras apagarse!

También puedes modificar el número máximo predeterminado de acciones registradas mediante el código:

```

53  const int act_max=10000; //Default 20 action,4 the Angle of servo
54  int act[act_max][4]; //Only can change the number of action
55  int num=0,num_do=0;

```

Output

```

cmd /c IF 0==1 COPY /y "C:\\Users\\11235\\AppData\\Local\\Arduino15\\packages\\esp32\\tools\\openocd
cmd /c IF 0==1 COPY /y "C:\\Users\\11235\\AppData\\Local\\Arduino15\\packages\\esp32\\hardware\\esp3
cmd /c IF 0==1 COPY /y "C:\\Users\\11235\\AppData\\Local\\Arduino15\\packages\\esp32\\hardware\\esp3

Using library ESP32Servo at version 3.0.5 in folder: C:\\Users\\11235\\Documents\\Arduino\\libraries\\ESP3
"C:\\Users\\11235\\AppData\\Local\\Arduino15\\packages\\esp32\\tools\\riscv32-esp-elf-gcc\\esp-2021r
Sketch uses 251604 bytes (19%) of program storage space. Maximum is 1310720 bytes.
Global variables use 174428 bytes (53%) of dynamic memory, leaving 153252 bytes for local variables.

```

Tras compilar el código en el IDE de Arduino, comprueba el registro de salida. Cuando el número de acciones registradas se cambia a 10 000, solo ocupa el 53 % de la memoria dinámica, lo que significa que aún puedes registrar más acciones.

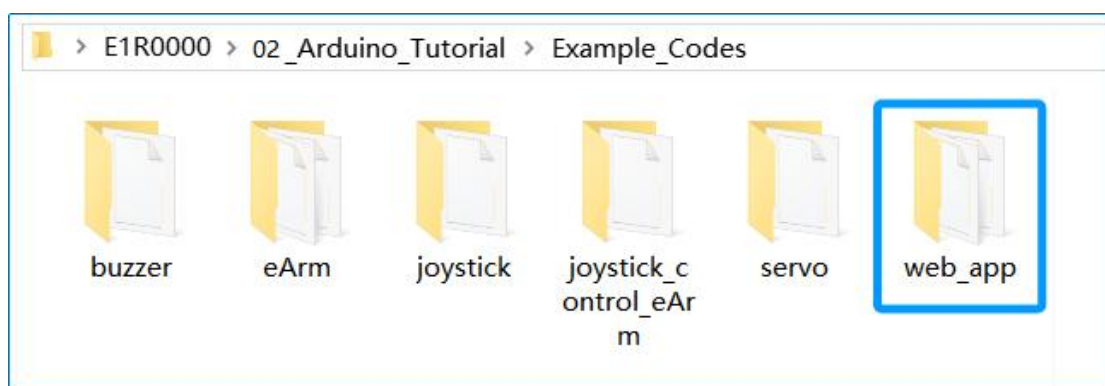
Explicación del código:

<code>eArm arm;</code>	Define una clase de brazo robótico.
<code>arm.ServoAttach(4,5,6,7);</code>	Los servos A, B, C y D del brazo robótico se conectan a los pines 4, 5, 6 y 7 de la placa de control, respectivamente.
<code>arm.JoyStickAttach(0,1,10,2,3,8);</code>	Los pines X, Y y Z del joystick izquierdo están conectados a los pines 0, 1 y 10 de la placa de control, respectivamente. Los pines X, Y y Z del joystick derecho están conectados a los pines 2, 3 y 8 de la placa de control, respectivamente.
<code>int act[x][y];</code>	Define una matriz bidimensional de enteros, donde x es el parámetro de fila y y es el parámetro de columna.
<code>arm.down(speed);</code>	El brazo robótico se balancea hacia abajo, siendo speed la velocidad del balanceo.
<code>arm.up(speed);</code>	El brazo robótico se mueve hacia arriba, siendo speed la velocidad del movimiento.
<code>arm.left(velocidad);</code>	El brazo robótico gira hacia la izquierda, siendo «speed» la velocidad de giro.
<code>arm.right(velocidad);</code>	El brazo robótico gira hacia la derecha, siendo «speed» la velocidad de giro.
<code>arm.clawClose(velocidad);</code>	La pinza del brazo robótico se cierra, siendo «speed» la velocidad de oscilación.
<code>arm.clawOpen(velocidad);</code>	La pinza del brazo robótico se abre, siendo «speed» la velocidad de oscilación.
<code>void turnUD(void);</code>	El joystick controla el movimiento del brazo robótico hacia arriba y hacia abajo.

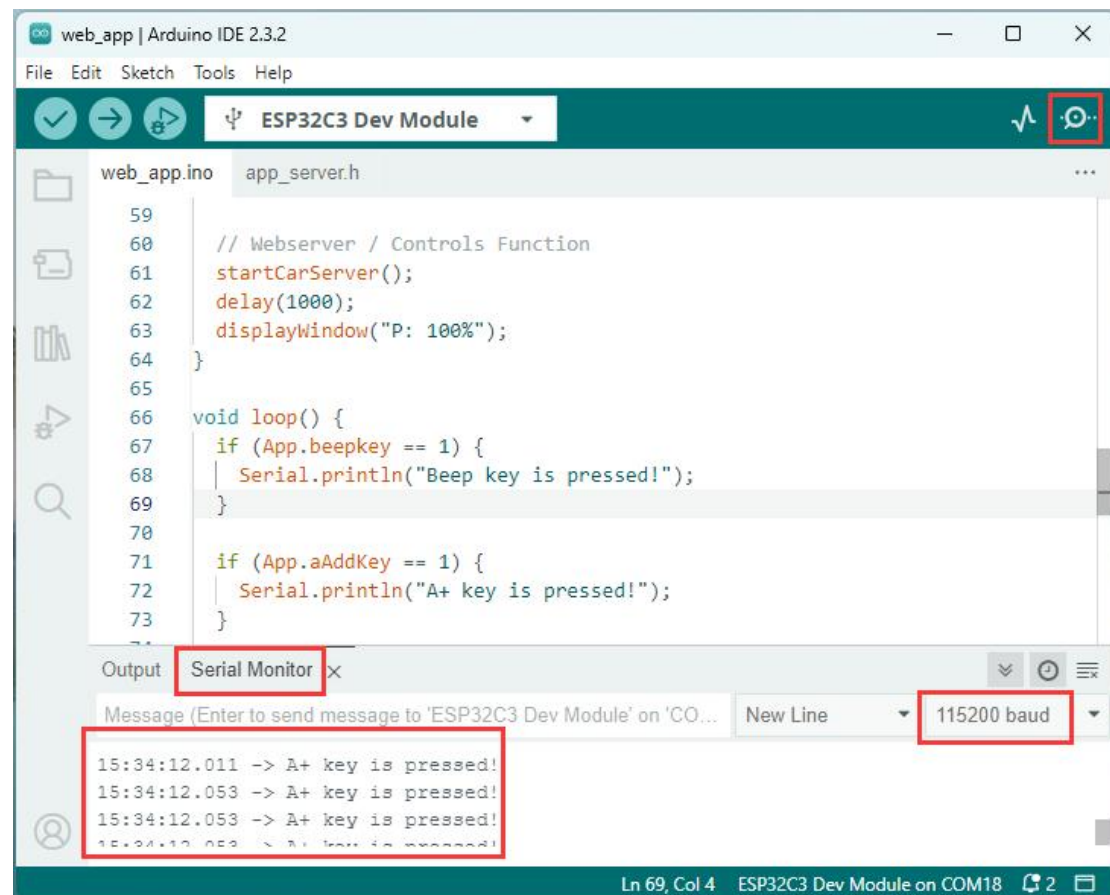
<code>void turnLR(void);</code>	El joystick controla el brazo robótico para que gire hacia la izquierda y hacia la derecha.
<code>void claw(void);</code>	El joystick controla la pinza del brazo robótico para que se abra y se cierre.
<code>void date_processing(int *x, int *y);</code>	Procesa los datos de los ejes X e Y del joystick y lee los valores máximos de oscilación de los ejes.
<code>void buzzer(int H, int L);</code>	Función del zumbador para los botones del eje Z del joystick. H es el tiempo que el zumbador está activado (en microsegundos); L es el tiempo que el zumbador está desactivado (en microsegundos).
<code>arm.JoyStickL.read_z();</code>	Lee el valor booleano del eje Z del joystick izquierdo.
<code>arm.JoyStickR.read_z();</code>	Lee el valor booleano del eje Z del joystick derecho.
<code>void Re_action(void);</code>	Registra las acciones actuales del brazo robótico y las guarda en una matriz bidimensional.
<code>void Ex_action(void);</code>	Ejecuta todas las acciones registradas en la matriz bidimensional.
<code>void executionAction(int *angle, int speed);</code>	Ejecuta una sola acción, donde *angle es un puntero y speed es la velocidad de ejecución.
<code>arm.JoyStickL.read_x();</code>	Lee los datos del eje X del joystick izquierdo. Valor de retorno: 0-4059.
<code>arm.JoyStickL.read_y();</code>	Lee los datos del eje Y del joystick izquierdo. Valor devuelto: 0-4059.
<code>arm.JoyStickR.read_x();</code>	Lee los datos del eje X del joystick derecho. Valor devuelto: 0-4059.
<code>arm.JoyStickR.read_y();</code>	Lee los datos del eje Y del joystick derecho. Valor devuelto: 0-4059.

1.5.5 Leer el valor de la aplicación web

Sigue las instrucciones del capítulo 1.5.1 y sube el código «web_app» a la placa de control esp32:



Sigue las instrucciones del capítulo «**Guía de montaje y uso**» para conectar el «eArm» a la red wifi y utilizar la aplicación web; a continuación, abre el monitor serie del IDE de Arduino y haz clic en el botón de la aplicación web. El monitor serie mostrará la cadena correspondiente al botón:



Explicación del código:

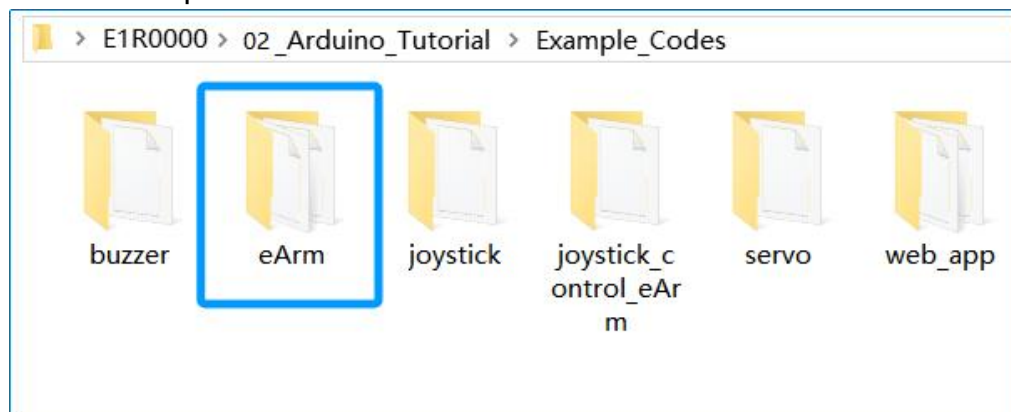
<code>#include "app_server.h"</code>	Incluye el archivo de cabecera «app_server.h» de la misma carpeta.
<code>bool ap = 1;</code>	Define una variable booleana «ap» y le asigna el valor 1.
<code>const char* ssid = "eArm";</code>	Define un puntero a carácter constante y asígnale un valor.
<code>int channel = 11;</code>	Define una variable entera «channel» y le asigna el valor 11.
<code>WiFi.mode(WIFI_STA);</code>	Configurar el Wi-Fi en modo «estación».
<code>WiFi.begin(ssid, password);</code>	Establece el nombre y la contraseña de la red Wi-Fi, e inicia la conexión Wi-Fi.
<code>while(condición) {...}</code>	Sentencia de bucle condicional «while»: si la condición es verdadera, el bucle ejecuta las

	instrucciones entre llaves; en caso contrario, el bucle se interrumpe.
<code>WiFi.status();</code>	Lee el estado de la conexión Wi-Fi.
<code>WiFi.localIP();</code>	Lee la dirección IP de la red wifi en modo «estación».
<code>WiFi.mode(WIFI_AP);</code>	Configurar el wifi en modo AP.
<code>WiFi.softAP(ssid, contraseña, canal, oculto, maxconexiones);</code>	En modo AP, configura el nombre de la red wifi, la contraseña, el canal, si se oculta el nombre y el número máximo de conexiones.
<code>WiFi.softAPIP();</code>	Lee la dirección IP del wifi en modo AP.
<code>startCarServer();</code>	Inicia el servidor web.
<code>displayWindow("P: 100%");</code>	Muestra la cadena en la interfaz de la página web.
<code>if(condición){ ... }</code>	Si la condición es verdadera, se ejecutan las instrucciones entre llaves; de lo contrario, no se ejecutan.
<code>App.beepkey</code>	Valor de la variable de estado del botón del zumbador en la interfaz de la página web.

1.5.6 Uso del brazo robótico

Este código de ejemplo es el código fuente completo del firmware de fábrica. Al cargarlo en la placa de control, se restablecerá la funcionalidad original de fábrica.

Sigue las instrucciones del capítulo 1.5.1 para cargar el código «eArm» en la placa de control esp32:



Una vez cargado el código eArm, ya puede controlar este brazo robótico tal y como se describe en la sección anterior: [Guía de montaje y uso > Uso del brazo robótico](#).

Explicación del código:

<code>TaskHandle_t TASK_HandleOne = NULL;</code>	Crear un identificador de hilo.
<code>xTaskCreate(...);</code>	Inicializar el hilo.
<code>void TASK_ONE(void* param) { ... }</code>	Función del hilo, que se ejecuta en paralelo con la función loop().

2

Grabar el firmware

Proporcionar el «programa de restablecimiento de fábrica» del dispositivo directamente a los usuarios en forma de archivo de firmware (por ejemplo, .bin o .hex), lo que les permite flashearlos en el dispositivo sin necesidad de configurar un entorno de desarrollo de Arduino, completando así el proceso de restauración.

1. Problemas con el enfoque tradicional

Normalmente, restaurar un dispositivo mediante Arduino requiere que los usuarios:

- ▶ Instalar el IDE de Arduino o herramientas de desarrollo relacionadas.
- ▶ Descargar el código fuente (archivo .ino), lo que puede requerir la configuración de bibliotecas adicionales y dependencias.
- ▶ Compilarlo y subirlo al dispositivo.

Este proceso supone una gran barrera para los usuarios sin conocimientos técnicos y puede fallar debido a problemas relacionados con el entorno.

2. Ventajas de proporcionar el firmware directamente

Proceso simplificado para el usuario:

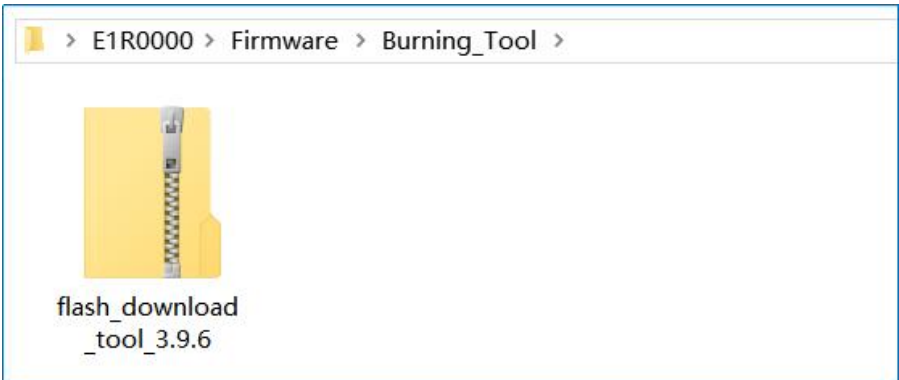
- ▶ Proporciona un archivo de firmware precompilado (por ejemplo, firmware.bin), lo que permite a los usuarios flashearlos con una herramienta sencilla (por ejemplo, un flasheador serie) en un solo paso.
- ▶ No es necesario instalar el IDE de Arduino, resolver errores de compilación ni gestionar conflictos entre bibliotecas.

Casos de uso:

- ▶ Restablecer la configuración de fábrica de un dispositivo que no funciona correctamente.
- ▶ Actualización masiva del mismo firmware en varios dispositivos (mejora la eficiencia).

2.1 Herramienta de grabación

La herramienta de grabación se encuentra en la siguiente carpeta:



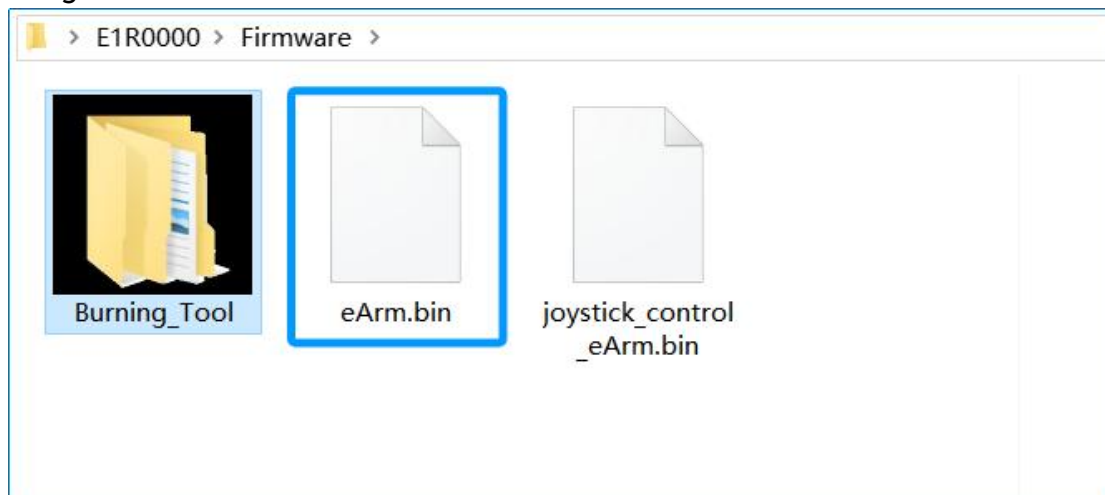
También puedes descargar la última versión de la herramienta de flasheo desde la página web oficial:

<https://www.espressif.com/en/support/download/other-tools>

Flash Download Tools				Expand all +
Please refer to Flash Download Tool User Guide to download this tool.				
Title	Platform	Version	Release Date ▾	Download
+ Flash Download Tools	HTML	latest	2024.12.18	

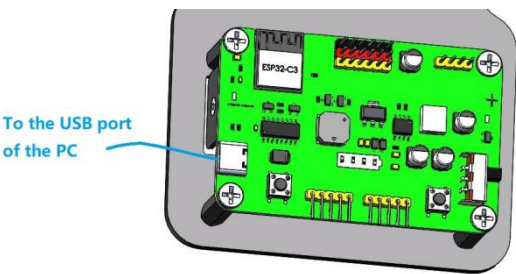
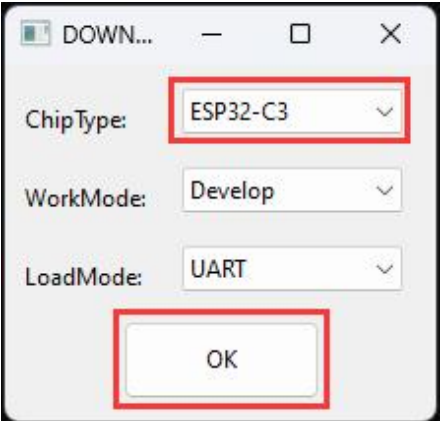
2.2 Firmware de eArm (firmware de fábrica)

Las funciones de este firmware son exactamente las mismas que las funciones de fábrica del brazo robótico. Además, tiene la misma funcionalidad que el código del capítulo 1.5.6. Si se pierden las funciones de fábrica, puedes utilizar este firmware para restaurar la configuración de fábrica. El firmware se encuentra en el siguiente archivo:

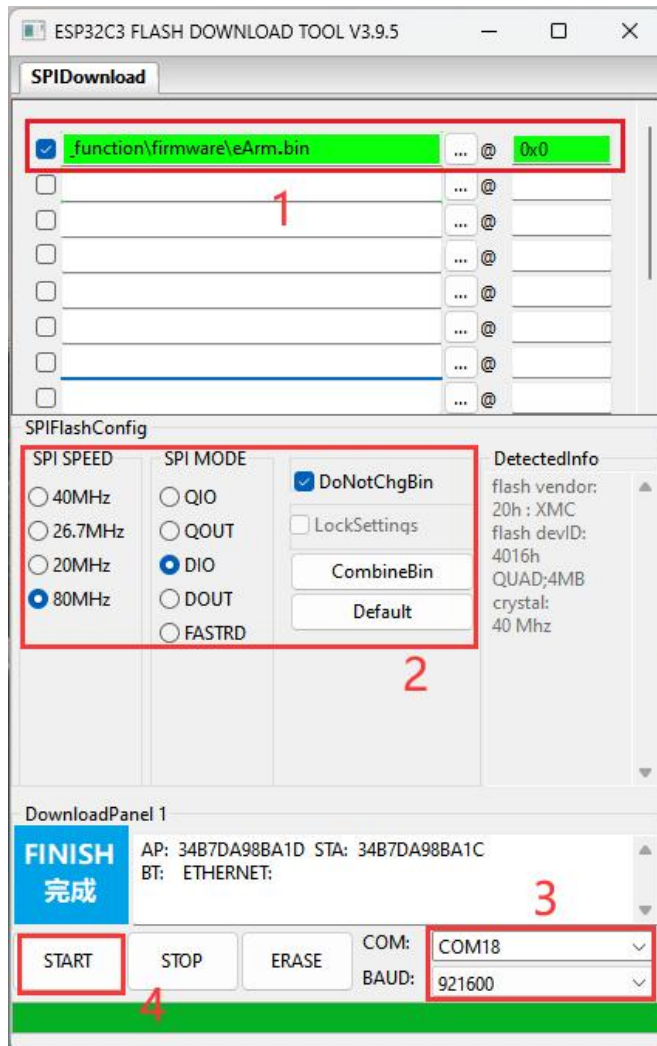


¡ Este firmware se genera a partir del código del capítulo 1.5.6!

2.2.1 Grabar el firmware de eArm

Utiliza el cable USB para conectar el eArm al ordenador:	Inicia la herramienta de grabación:
	

Selecciona el firmware que deseas grabar e introduce correctamente la dirección de grabación.



El controlador CH340 debe estar instalado, la batería debe estar correctamente conectada y el interruptor de encendido del eArm debe estar activado; de lo contrario, no se

1: Marca una de las opciones de grabación «», a continuación haz clic en «» y selecciona el archivo de firmware «**eArm.bin**» que se proporciona en nuestro tutorial. A continuación, introduce correctamente la dirección de grabación (**0x0**) después de «@»:

Nombre del archivo	Dirección de grabación
eArm.bin	0x0

2: Selecciona **80 MHz** en «SPI SPEED», «**DIO**» en «SPI MODE» y marca las casillas «» y «**DotNotChgBin**».

3: Selecciona el puerto serie para la placa de control. En este caso, es el COM18 (selecciona el puerto según el que tu ordenador haya asignado realmente a este dispositivo). Ajusta la velocidad de transmisión a **921 600**. Si la grabación falla, puedes reducir la velocidad de transmisión, por ejemplo, a **115 200**.

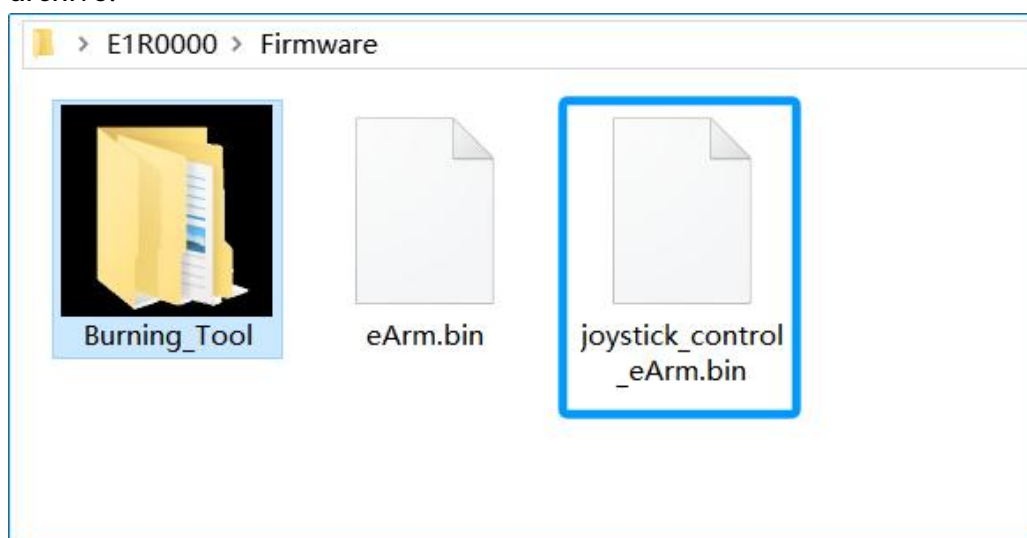
4: Haz clic para iniciar la grabación del firmware.

Una vez completada la grabación, ¡puedes seguir la sección «[Montaje y guía de usuario](#)» para controlar el brazo robótico!

Nota: Una vez finalizada la actualización del firmware, debes pulsar el botón RST (Reset) o desconectar el cable USB y, a continuación, encender el dispositivo.

2.3 Firmware de control mediante joystick (con función de grabación)

Este firmware tiene las mismas funciones que el código del capítulo 1.5.4, con una capacidad de grabación de 20 acciones. El firmware se guarda en el siguiente archivo:

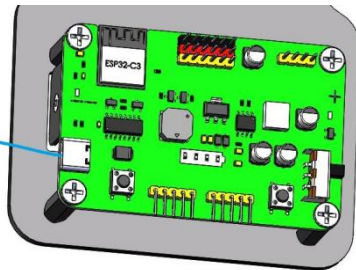


Este firmware se ha generado a partir del código del capítulo 1.5.4

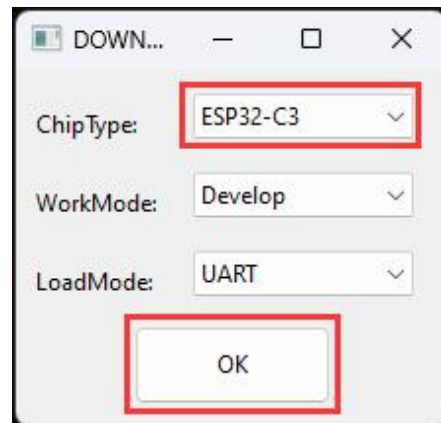
2.3.1 Grabar el firmware de control mediante joystick

Utiliza el cable USB para conectar el eArm al ordenador:

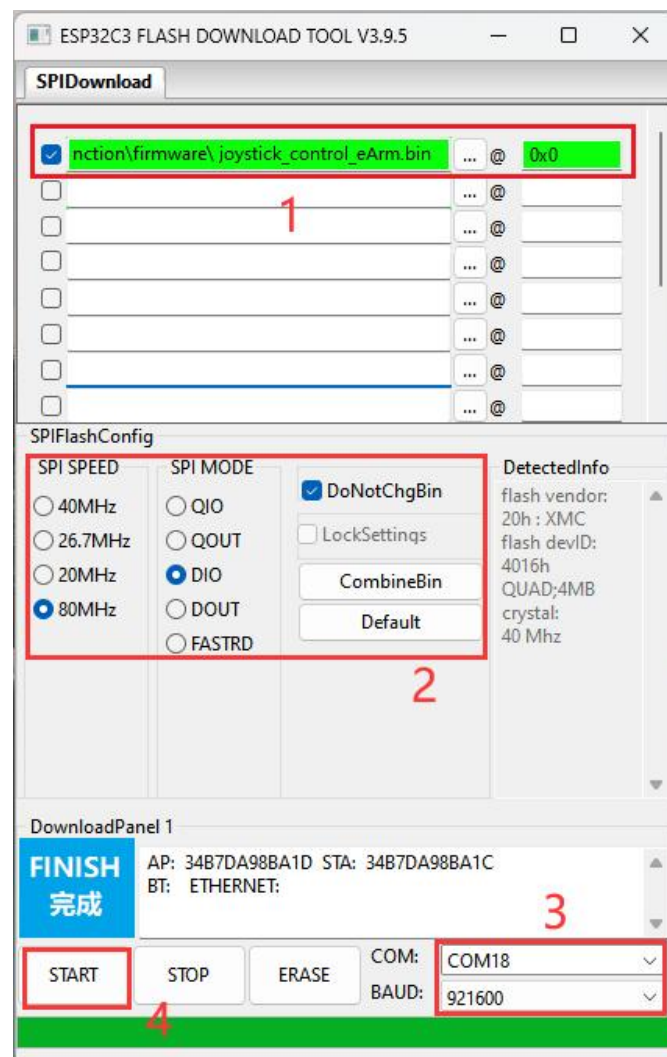
To the USB port
of the PC



Inicia la herramienta de grabación:



Selecciona el firmware que deseas grabar e introduce correctamente la dirección de grabación



Debe tener instalado el controlador CH340, la batería debe estar correctamente conectada y el interruptor de encendido del eArm debe estar activado; de lo contrario, no se

1: Marca una de las opciones de grabación «  », a continuación haz clic en «  » y selecciona el archivo de firmware «[joystick_control_eArm.bin](#)» que se proporciona en nuestro tutorial. A continuación, introduce correctamente la dirección de grabación (**0x0**) después de «@»:

Nombre del archivo	Dirección de grabación
joystick_control_eArm.bin	0x0

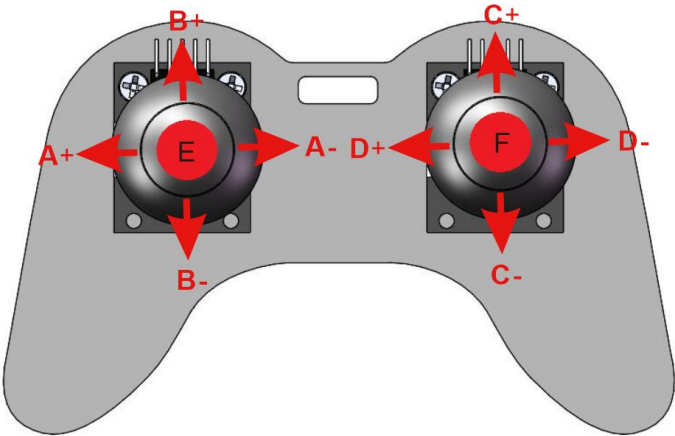
2: Selecciona **80 MHz** para «SPI SPEED», «**DIO**» para «SPI MODE» y marca la casilla «  » en «**DotNotChgBin**».

3: Selecciona el puerto serie para la placa de control. En este caso, es el COM18 (selecciona el puerto según el que tu ordenador haya asignado realmente a este dispositivo). Establece la velocidad de transmisión en **921 600**. Si la grabación falla, puedes reducir la velocidad de transmisión, por ejemplo, a **115 200**.

4: Haz clic para iniciar la grabación del firmware.

Nota: Una vez completada la actualización del firmware, debes pulsar el botón RST (Reset) o desconectar el cable USB y, a continuación, encender el dispositivo.

Control mediante joystick - Guía de funcionamiento del brazo robótico:

	
A+: Controla el A servo; el brazo robótico gira hacia la izquierda.	A-: Controla el servo A; el brazo robótico gira a la derecha.
B+: controla el B servo; el brazo trasero del brazo robótico se balancea hacia delante.	B-: Controla el servomotor B; el brazo trasero del brazo robótico se balancea hacia atrás.
C+: Controla el servomotor C; el antebrazo del brazo robótico se balancea hacia delante.	C-: Controla el servomotor C; el antebrazo del brazo robótico se balancea hacia atrás.
D+: Controla el servo D; la pinza del brazo robótico se abre (se afloja).	D-: Controla el servomotor D; la pinza del brazo robótico se cierra (se aprieta).

E: Grabar acción – Pulsa el botón E una vez para grabar una acción (se pueden grabar hasta 20 acciones seguidas). Cada vez que se pulsa, se oye un pitido corto. Cuando se alcanza la capacidad máxima de grabación, el zumbador emite un pitido largo.	F: Reproducción — Tras grabar la secuencia, pulsa una vez el botón F para iniciar la reproducción en bucle de las acciones grabadas de forma continua. El zumbador emitirá un pitido y el brazo repetirá las acciones en bucle. Para salir del bucle, mantén pulsado el botón F (pulsación prolongada); el zumbador emitirá un pitido largo para indicar que la reproducción se ha detenido.
---	--

3

Preguntas frecuentes

3.1 No se reconocen los puertos serie USB

- 🔧 ¿Utilizas un cable USB con función de comunicación de datos?
- 🔧 ¿Está bien conectado el cable USB?
- 🔧 ¿Está instalado el controlador USB CH340?

3.2 El brazo robótico no funciona

- 👉 ¿Hay que activar el interruptor de encendido durante la instalación para inicializar el ángulo del servo?
- 🔧 ¿Hay suficiente carga en la batería?

3.3 Otros problemas

- 🔧 ¿Por favor, comprueba si el montaje es correcto?
- 🔧 Compruebe si la batería tiene suficiente carga.
- 🔧 Compruebe si la batería utilizada cumple con las especificaciones.

4

Contacto

Si tiene algún problema técnico o desea enviarnos sus comentarios, no dude en ponerse en contacto con nosotros



support@siyeenove.com



<https://siyeenove.com>